


Set

```
const myarray = [1, 2, 3, 4]
```

```
const myset = new Set(myarray)
```

```
console.log(myset)
```

Output: Set(4) {1, 2, 3, 4}

```
const myarray = [1, 2, 3, 4, 5, 5, 4, 3, 2, 1]
```

```
const myset = new Set(myarray)
```

```
console.log(myset)
```

Output: Set(5) {1, 2, 3, 4, 5}

```
const uniqueArray = [...myset]
```

→ spread Operator

```
console.log(uniqueArray)
```

Output: Set(5) {1, 2, 3, 4, 5}

```
myset.add(6)
```

```
console.log(myset)
```

Output: Set(6) {1, 2, 3, 4, 5, 6}

myset.add('6')

output: set(7){1, 2, 3, 4, 5, 6, '6'}

myset.add([7, 8])

output: set(7){1, 2, 3, 4, 5, 6, '6', [7, 8]}

myset.delete(5)

output: set(7){1, 2, 3, 4, 6, '6', [7, 8]}

myset.clear()

output: set(0){ }

Map

```
const myMap = new Map([['FirstName', 'John'],  
                        ['lastName', 'Doe']]);
```

```
console.log(myMap)
```

OP: { 'FirstName' => 'John',
 'lastName' => 'Doe' }

We had objects then why maps?

```
const myobject = { };
```

```
const a = { };
```

```
const b = { };
```

```
myobject[a] = 10;
```

```
console.log(myobject)
```

Op: (`'[object object]'`: 10)

it was an object but has converted
into a string

```
myobject[a] = 10;
```

```
myobject[b] = 20;
```

```
console.log(myobject)
```

Op: (`'[object object]'`: 20)

To use many objects as key: value pairs we use
Map.

```
const myMap = new Map([[a, 10], [b, 20]])
```

```
console.log(myMap)
```

OP: Map(2) { 3 => 10, 1 => 20 }

```
myMap.set('a', 30);
```

```
console.log(myMap)
```

OP: Map(3) { 3 => 10, 1 => 20, 'a' => 30 }

```
myMap.delete('a')
```

```
console.log(myMap.has('a'))
```

OP: False

```
console.log(myMap.has(10))
```

OP: False

map searches by keys not values.

c.l (myMap.size);

Error handling

we use try => lets you check the code for errors

catch => lets you execute some code if error occurs

throw => let you customize code for different errors

Finally => lets you execute code regardless of the error.

try {

console.log('start of try block')

add;

console.log('End of try block')

}

catch {

console.log('Error has Occurred:', e.message)

}

Finally {

console.log('We are inside finally')

}

Op: Start of the try block

Finally

We are inside Finally

Normal vs Arrow Func

Function normalGreet (name)

{ console.log('Hello', name);

}

const arrowGreet = (name) => { console.log('Hello', name); }

{ console.log('Hello', name); }

→ if its inline then this bracket can be discarded.

normalgreet('ssshyhe')

Arrowgreet('ssshyhe')

OP: ssshhyhe
ssshyhe